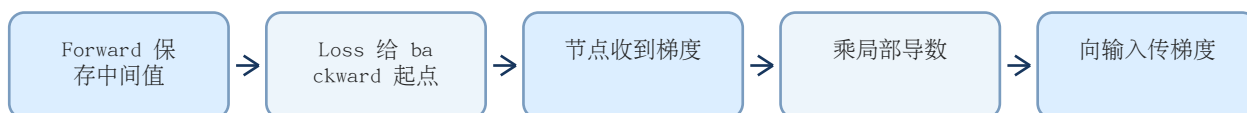


第23讲 实验设计

把节点级 Backward 真正跑起来

核心概念：实验设计

本讲把 Chain Rule 变成真正可执行的 backward 规则。目标是：看到一个计算图节点时，知道它在 Forward 保存什么，在 Backward 收到什么梯度、乘哪个局部导数、把梯度传给哪些输入。



建议学习方式：先掌握四类标量节点规则 -> 再手工走完整图 -> 最后用数值差分检查每个参数梯度。

一、实验目标

逐个验证 Add、Multiply、Square、ReLU 的局部 backward 规则。

对一个完整标量神经元计算图执行 Forward 和手工 Backward。

用 central difference 检查 w 、 b 、 x 的手工梯度。

观察 ReLU 关闭时 Loss 很大但参数梯度仍为0。

验证共享变量的多个路径贡献必须累加。

用一次最小参数更新理解 backward 与 optimizer 的接口关系。

工具: Python + NumPy + Matplotlib。仍然不依赖 PyTorch Autograd。

二、实验1：局部节点规则

统一设后方传来的梯度 $g_z=4$ ，取 $a=2, b=-3$ 。

节点	预期结果
Add	$g_a=4, g_b=4$
Multiply	$g_a=-12, g_b=8$
Square (a)	$g_a=16$
ReLU (a=2)	$g_a=4$
ReLU (b=-3)	$g_b=0$

目的不是背数字，而是确认：同一个 incoming gradient 经过不同算子后会按照不同局部导数发生变化。

三、实验2：完整计算图 Forward + Backward

$$m=w \times x$$

$$z=m+b$$

$$a=\text{ReLU}(z)$$

$$d=a-\text{target}$$

$$L=d^2$$

取 $x=2, w=3, b=-1, \text{target}=4$ 。

阶段	预期
Forward	$m=6, z=5, a=5, d=1, L=1$
Backward	$dd=2, da=2, dz=2, db=2, dw=4, dx=6$

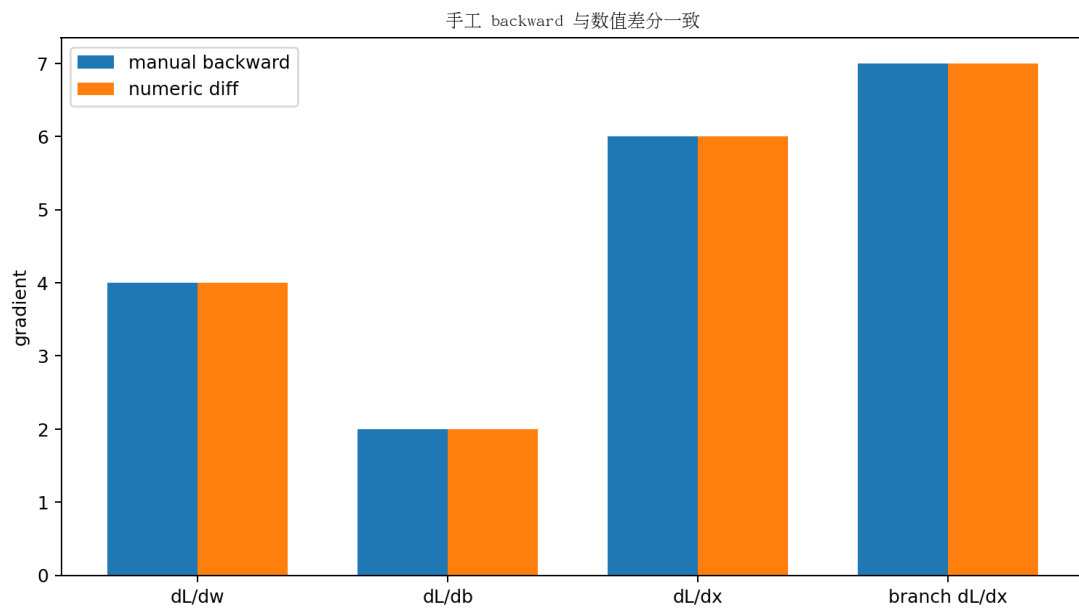
代码会把每一个中间值和每一个梯度都打印出来。

四、实验3：Gradient Check

$$dL/dq \approx [L(q+\epsilon) - L(q-\epsilon)] / (2\epsilon)$$

分别对 $q=w, b, x$ 做数值差分。验收标准：

量	手工	数值	允许误差
dL/dw	4	约4	$<1e-6$
dL/db	2	约2	$<1e-6$
dL/dx	6	约6	$<1e-6$



五、实验4：ReLU 截断

把 w 改成 -3 :

$$z=-7, \text{ReLU}(z)=0, \text{Loss}=16$$

尽管 Loss 比之前更大，但：

$$d \text{ReLU}/dz = 0$$

所以：

$$dL/dw=0$$

这个实验要求你把“Loss 大小”和“Gradient 大小”彻底分开理解。

六、实验5：共享变量与梯度累加

$L=x^2+3x, x=2$

路径	贡献
$x \rightarrow x^2 \rightarrow L$	4
$x \rightarrow 3x \rightarrow L$	3
总 dL/dx	7

代码用数值差分再次验证7。

科研连接：实现要求

如果以后自己写 micrograd 一类的小型自动微分引擎，变量的 grad 应该做 += 累加，而不是简单覆盖。

七、实验6：一次参数更新

使用实验2的手工梯度：

$$dw=4, db=2$$

设：

$$lr=0.05$$

更新：

$$w: 3 \rightarrow 2.8$$

$$b: -1 \rightarrow -1.1$$

重新 Forward：

$$\text{Loss: } 1.00 \rightarrow 0.25$$

数学直觉：这里验证的不是优化器优劣

这里只检查 backward 产生的梯度是否能够被最简单的参数更新规则使用。正式 Gradient Descent 会在后面系统学习。

八、结果记录表

检查项	实际结果	是否通过
Multiply 节点 g_a/g_b	-12 / 8	是
完整图 dL/dw	4	是
完整图 dL/db	2	是
完整图 dL/dx	6	是
数值差分最大误差	约 $1e-10$	是
ReLU 关闭后的 dL/dw	0	是
共享变量总梯度	7	是
一步更新 Loss	1 $\rightarrow$ 0.25	是

九、实验结论模板

具体例子：结论1

Backward 可以被分解为节点级局部规则：后方梯度乘本节点局部导数。

具体例子：结论2

完整网络梯度不需要一次写出全局导数公式，只要按计算图逆序组合局部规则。

具体例子：结论3

一个变量有多条后续路径时，所有路径贡献必须累加。

科研连接：结论4

梯度存在与是否更新是两回事；optimizer 只更新被定义为可训练的参数。

科研连接：结论5

数值差分可以作为手工/自定义 backward 的 sanity check。

十、运行方法

```
cd deep_learning_lesson23/final
python lesson23_computational_graph_experiment.py
```

```
# 查看:
# 第23讲_实验实际输出.txt
# figures/lesson23_forward_backward_graph.png
# figures/lesson23_local_node_rules.png
# figures/lesson23_gradient_check.png
```

如果缺少依赖:

```
pip install numpy matplotlib
```

CPU 即可, 无需 GPU。